

## Проект News Engine

### # NEWS ENGINE — ПРОМТ-ТЗ ДЛЯ OPENCODE

Ты — ведущий software architect и senior Python engineer.  
Спроектируй и реализуй прототип модульного self-hosted News Engine.

Главная цель:

создать быстрый, дешёвый по ресурсам и масштабируемый pipeline для сбора игровых и других новостей, который максимально использует существующее open-source ПО и минимально зависит от LLM/AI-агентов.

НЕ начинай с написания большого собственного движка.

Сначала исследуй выбранные OSS-компоненты, их API/CLI/Docker-возможности, лицензии и производительность.

Используй существующие проекты как строительные блоки.

---

### # 1. ГЛАВНЫЙ ПРИНЦИП

Архитектура должна быть:

SOURCE → ARTICLE → FACTS → STORY → TRANSLATION → EDITORIAL → QUALITY  
→ CONTENT → ASSETS → OUTPUT

LLM НЕ должна быть обязательным этапом каждой новости.

Предпочтительный порядок:

1. deterministic code
2. существующая ML/NLP модель
3. локальная маленькая модель
4. LLM только как fallback для сложных случаев

Не использовать AI-agent loop для обычной обработки.

AI-агент может использоваться только разработчиком/оператором для изменения pipeline, но не должен быть runtime-зависимостью News Engine.

---

### # 2. ЦЕЛИ

Система должна уметь:

- собирать новости из RSS/Atom;
- получать полный текст статьи;

- нормализовать данные;
- удалять дубликаты;
- объединять несколько публикаций об одном событии;
- определять важность;
- извлекать факты;
- определять сущности;
- переводить новости на русский;
- сохранять игровые термины;
- формировать естественный русский текст;
- проверять качество русского текста;
- оценивать вероятность AI-generated текста;
- собирать качественные изображения;
- удалять дубликаты изображений;
- оценивать соответствие изображения новости;
- формировать единую Story;
- выдавать Story в JSON;
- генерировать короткий пост;
- генерировать карточку;
- генерировать VK carousel;
- генерировать Telegram post;
- генерировать горизонтальное видео;
- использовать HyperFrames/HTML/CSS как основу визуального шаблона;
- позволять пользователю вручную запросить:
  - "Вот новость про DLSS 5 — сделай маленький пост и карточку";
- позволять запросить:
  - "Дай последние 10 новостей";
- позволять сформировать:
  - "Последние новости для Telegram";
- позволять сформировать:
  - "10 новостей + 10 карточек + видео".

---

### # 3. КЛЮЧЕВОЙ ОБЪЕКТ — STORY

Вся система должна строиться вокруг единого нормализованного объекта Story.

Создать строгую JSON Schema/Pydantic schema.

Минимально:

```
{
  "story_id": "...",
  "category": "games",
  "importance": 0-100,

  "sources": [],
```

```
"facts": [],  
"entities": [],  
"original_language": "en",  
"translation": {},  
"content": {},  
"quality": {},  
"assets": [],  
"render": {},  
"created_at": "...",  
"updated_at": "..."  
}
```

Story должна быть независима от конкретного output.

VK, Telegram, Card и Video являются производными от Story.

---

#### # 4. SOURCE

Source:

```
{  
  "source_id": "...",  
  "url": "...",  
  "publisher": "...",  
  "author": "...",  
  "published_at": "...",  
  "language": "...",  
  "title": "...",  
  "feed_url": "...",  
  "retrieved_at": "...",  
  "raw_hash": "...",  
  "content_hash": "..."  
}
```

Обязательно сохранять provenance.

Каждый факт должен иметь source\_id.

Нельзя превращать непроверенный текст в факт без указания источника.

---

## # 5. FACT

Fact:

```
{
  "fact_id": "...",
  "story_id": "...",
  "source_id": "...",
  "text": "...",
  "type": "release_date",
  "confidence": 0.0,
  "entities": [],
  "evidence": {
    "quote": "...",
    "url": "..."
  }
}
```

Не надо генерировать факты из воздуха.

Если разные источники противоречат друг другу — сохранять оба утверждения и создавать conflict flag.

---

## # 6. COLLECTOR

## Основной кандидат

Miniflux.

<https://github.com/miniflux/v2>

Использовать как RSS/Atom ingestion layer.

Проверить:

- API;
- Docker;
- webhook;
- polling;
- categories;
- tags;
- read/unread;

- database;
- performance.

## ## Альтернатива А

FreshRSS.

<https://github.com/FreshRSS/FreshRSS>

## ## Альтернатива В

Прямой Python RSS collector:

feedparser / aiohttp / httpx.

## ## Прототипирование

Реализовать интерфейс:

SourceAdapter

Например:

```
class SourceAdapter:
    async def get_items(...)
    async def mark_processed(...)
    async def health_check(...)
```

Сделать минимум:

MinifluxAdapter  
DirectRSSAdapter

FreshRSSAdapter можно оставить экспериментальным.

---

## # 7. ARTICLE EXTRACTION

Основной кандидат:

Trafilatura

<https://github.com/adbar/trafilatura>

Использовать для получения:

- title;

- text;
- author;
- date;
- metadata;
- categories.

## ## Альтернативы

A:  
Trafilatura

B:  
readability-lxml

C:  
newspaper4k

D:  
jusText

Создать ExtractionBenchmark.

Для одного набора URLs измерять:

- success rate;
- extracted text length;
- boilerplate contamination;
- execution time;
- RAM;
- correctness title/date/author.

Не выбирать библиотеку "на глаз".

---

## # 8. NORMALIZATION / RUSSIAN NLP

Основной кандидат:

Natasha

<https://github.com/natasha/natasha>

Использовать для:

- sentence segmentation;
- tokenization;
- morphology;

- lemmatization;
- NER;
- fact extraction;
- normalization.

Особенно важно для:

- компаний;
- игр;
- людей;
- платформ;
- дат;
- студий;
- издателей;
- технологий.

Не использовать LLM там, где Natasha решает задачу.

---

## # 9. DEDUP

Сделать несколько уровней.

### ## Level 1

Exact hash.

### ## Level 2

Normalized title hash.

### ## Level 3

SimHash / MinHash.

### ## Level 4

Embedding similarity.

Не использовать LLM.

Параметры должны быть конфигурируемыми:

duplicate\_threshold  
cluster\_threshold  
time\_window

Benchmark:

- false duplicate;
- missed duplicate;
- CPU time;
- RAM.

---

## # 10. STORY CLUSTERING

Нужно объединять:

IGN:

"Exodus release date announced"

GameSpot:

"Exodus launches April 7, 2027"

PC Gamer:

"Mass Effect-like Exodus gets release date"

в одну Story.

### ## Вариант A — Allsikt

<https://github.com/sebastianaldrin/allsikt-engine>

Исследовать и переиспользовать его clustering approach/components.

Он использует комбинацию:

- embeddings;
- TF-IDF;
- entities;
- similarity;
- Union-Find.

### ## Вариант B

sentence-transformers + cosine similarity.

### ## Вариант C

multilingual-e5-small / bge-small / аналогичная маленькая embedding model.

### ## Вариант D

детерминированный:

title similarity  
+  
entity overlap  
+  
publication time  
+  
keyword overlap.

Создать ClusterBenchmark.

Метрики:

precision  
recall  
false merge  
false split  
latency  
RAM  
CPU

Для production выбрать лучший вариант по результатам реального gaming corpus.

---

## # 11. IMPORTANCE SCORE

Не использовать LLM в первом варианте.

Создать deterministic scoring engine.

Например:

importance =  
source\_weight  
+ number\_of\_independent\_sources  
+ freshness  
+ entity importance  
+ release\_date  
+ announcement  
+ major\_update  
+ platform relevance  
+ user-defined topic weight

Пример:

official publisher announcement = +20

multiple independent sources = +15  
release date = +15  
new game announcement = +15  
major platform release = +10  
minor patch = +3

Все коэффициенты конфигурируемые.

Создать:

ImportanceScorer

Позже добавить экспериментальный LLM scorer.

Benchmark:

deterministic  
vs  
small local model

---

## # 12. SUMMARY

Очень важно:

НЕ использовать LLM для каждой статьи.

### ## Variant A

TextRank / Summa

<https://github.com/summanlp/textrank>

Extractive summary.

### ## Variant B

sentence-transformers + extractive ranking.

### ## Variant C

маленькая локальная LLM.

Протестировать:

- скорость;
- качество;

- factuality;
- длина;
- RAM;
- GPU usage.

Первый production вариант должен работать без LLM.

LLM-summary — fallback/experimental.

---

## # 13. TRANSLATION

Основной вариант:

OPUS-MT + CTranslate2.

<https://github.com/Helsinki-NLP/Opus-MT>

<https://github.com/OpenNMT/CTranslate2>

Проверить:

- INT8;
- CPU inference;
- batching;
- GPU optional.

### ## Alternative A

Argos Translate:

<https://github.com/argosopentech/argos-translate>

Полностью локальный перевод.

### ## Alternative B

NLLB небольшого размера через CTranslate2.

### ## Alternative C

маленькая LLM через Ollama.

LLM НЕ является default translator.

Сделать TranslationAdapter:

```
translate(  
    text,  
    source_language,  
    target_language,  
    terminology=None  
)
```

---

## # 14. TERMINOLOGY ENGINE

Это обязательный модуль.

Создать YAML terminology database.

Например:

gaming.yml

shooter: шутер  
first-person shooter: шутер от первого лица  
third-person shooter: шутер от третьего лица  
open world: открытый мир  
early access: ранний доступ  
free-to-play: условно-бесплатная  
gameplay: геймплей  
publisher: издатель  
developer: разработчик  
battle royale: королевская битва  
roguelike: рогалик

tech.yml

ray tracing: трассировка лучей  
frame generation: генерация кадров

brands.yml

NVIDIA: NVIDIA  
DLSS: DLSS  
RTX: RTX  
Xbox: Xbox  
PlayStation: PlayStation  
Steam: Steam

Названия игр, компаний и технологий нельзя переводить без explicit mapping.

Terminology engine должен работать до/после MT.

Использовать protected placeholders:

{{TERM\_001}}

→ translation

→ restore.

---

## # 15. EDITORIAL RUSSIAN

Взять идеи из:

<https://www.skills.sh/vladimir-human/humanizer-ru/humanizer-ru>

<https://www.skills.sh/talkstream/ru-text/ru-text>

<https://www.skills.sh/talkstream/ru-text/ru-score>

Но НЕ делать эти skills runtime-зависимостью.

Исследовать их правила и реализовать необходимые deterministic проверки как Python module.

Создать:

RussianEditorialEngine

Он должен исправлять/проверять:

- кальки;
- бюрократический стиль;
- повторения;
- чрезмерные вводные конструкции;
- неестественные конструкции;
- неправильные тире/кавычки;
- избыточные слова;
- канцелярит;
- шаблонные AI-конструкции;
- слишком длинные предложения;
- повтор названия;
- лишние вводные фразы.

---

## # 16. RU-SCORE

Создать:

RussianQualityChecker

Output:

```
{
  "score": 0-100,
  "issues": [],
  "metrics": {
    "sentence_length": ...,
    "repetition": ...,
    "bureaucratic": ...,
    "calques": ...,
    "readability": ...
  }
}
```

Не делать бинарную систему.

Quality score должен быть объяснимым.

---

## # 17. AI-TEXT DETECTOR

Экспериментальный компонент:

<https://github.com/k1y0miiii/ru-ai-text-detector>

Проверить особенно:

rubert-tiny2 / v2

Использовать detector только как QA signal.

НЕ пытаться специально "обмануть детектор".

Если:

quality < threshold

или

ai\_probability > threshold

→ отправлять на repair stage.

---

## # 18. REPAIR STAGE

Первый вариант:

deterministic repair.

Второй:

small local LLM.

LLM вызывается только при необходимости.

Например:

```
if score < 75:  
    deterministic_repair()
```

```
if score still < 75:  
    local_llm_rewrite()
```

```
if score still < 75:  
    manual_review
```

---

## # 19. LOCAL LLM

Создать LocalLLMAdapter.

Поддержать:

Ollama

и OpenAI-compatible endpoint.

Модель должна быть конфигурируемой.

Проверить маленькие модели:

- Qwen small instruct;
- Gemma small;
- Phi small;
- другие актуальные модели с хорошим русским.

Не хардкодить модель.

LLM должна использоваться только для:

1. сложного summary;
2. сложной редакторской переработки;
3. спорной классификации;
4. восстановления структуры, если extraction плохой.

---

## # 20. IMAGE COLLECTION

Основной вариант:

SearXNG

<https://github.com/searxng/searxng>

Использовать image search API.

Query строить из:

game\_name  
+  
event  
+  
publisher  
+  
release/update/etc.

### ## Альтернативы

- Wikimedia Commons API;
- Bing/Google adapters;
- direct source images;
- official publisher pages.

Все источники должны быть адаптерами.

---

## # 21. IMAGE EXTRACTION / FILTERING

После получения 10–30 кандидатов:

1. HTTP validation
2. content-type
3. dimensions

4. filesize
5. image decode
6. duplicate detection
7. quality score
8. semantic relevance

---

## # 22. IMAGE DEDUP

Использовать:

<https://github.com/JohannesBuchner/imagehash>

Проверить:

- pHash;
- dHash;
- wHash;
- crop-resistant hash.

Не использовать LLM.

---

## # 23. IMAGE QUALITY

Deterministic score:

resolution  
+  
aspect ratio  
+  
sharpness  
+  
compression  
+  
face detection  
+  
text density

Не брать маленькие изображения.

Настроить minimum resolution.

---

## # 24. IMAGE RELEVANCE

## ## Variant A

CLIP:

<https://github.com/openai/CLIP>

## ## Variant B

SigLIP.

## ## Variant C

маленькая локальная vision model.

Image-text similarity.

Например:

Story:

"NVIDIA представила DLSS 5"

Candidate:

NVIDIA GPU

DLSS screenshot

Cyberpunk screenshot

random NVIDIA logo

CLIP должен ранжировать соответствующие изображения выше.

---

## # 25. IMAGE FOCUS / CARD LAYOUT

Цель:

текст карточки не должен перекрывать главный объект изображения.

Первый вариант должен быть БЕЗ LLM.

Pipeline:

image

↓

resize

↓

saliency  
↓  
face/object detection  
↓  
edge/text density  
↓  
safe zones  
↓  
layout optimizer

Использовать:

OpenCV  
Pillow  
CLIP  
OCR

При наличии подходящего OSS-компонента для saliency/object detection — исследовать и подключить.

---

## # 26. LAYOUT ENGINE

Карточка должна иметь:

- category pill;
- крупный headline;
- summary/body;
- publisher;
- card number;
- image;
- branding.

Например:

|                              |       |
|------------------------------|-------|
| [ИГРЫ]                       | 01/10 |
| EXODUS ВЫЙДЕТ В 2027 ГОДУ    |       |
| Разработчики назвали дату... |       |
| IMAGE                        |       |
| GLFire                       |       |

Headline должен формироваться из первых слов Content.

Headline не должен быть отдельным несвязанным AI-generated title.

---

## # 27. CONTENT MODEL

Story → Content.

Content:

```
{
  "headline": "...",
  "deck": "...",
  "body": "...",
  "telegram": "...",
  "vk": "...",
  "card": {
    "headline": "...",
    "text": "..."
  }
}
```

Важно:

карточка и текст поста должны быть связаны.

Если в одном посте 10 новостей:

каждая Story получает:

01/10

02/10

...

10/10

А итоговый VK/Telegram post содержит соответствующий текстовый блок для каждой Story.

---

## # 28. SINGLE NEWS MODE

Комбайн должен принимать:

"Вот новость про DLSS 5.

Сделай маленький пост и карточку."

Pipeline:

```
input text/url
↓
extract
↓
facts
↓
normalize
↓
translate if needed
↓
editorial
↓
quality
↓
image
↓
card
↓
output
```

Не запускать полный news collection pipeline.

---

## # 29. BATCH MODE

Input:

"Собери последние новости."

Pipeline:

```
sources
↓
articles
↓
dedup
↓
clusters
↓
importance
↓
top N
↓
translation
```

↓  
editorial  
↓  
assets  
↓  
content  
↓  
cards  
↓  
VK/TG/video

---

### # 30. TELEGRAM OUTPUT

Создать Telegram renderer.

Он получает Content и возвращает Markdown/HTML.

Например:

10 news items.

Никаких повторных LLM-вызовов.

---

### # 31. VK OUTPUT

Создать VK renderer.

Поддержать:

- text;
- image;
- carousel;
- numbered cards;
- hashtags;
- links.

---

### # 32. CARD RENDERING

Использовать HTML/CSS как основной формат.

Не делать Photoshop обязательным runtime dependency.

Варианты:

A:

HTML/CSS → screenshot

B:

Pillow

C:

SVG → PNG

Benchmark по:

- rendering speed;
- typography;
- predictability;
- ease of templating.

HTML/CSS желательно сделать основным.

---

## # 33. VIDEO

Использовать HyperFrames:

<https://github.com/heygen-com/hyperframes>

Один и тот же Story должен использоваться для:

Card

и

Video.

Card:

HTML/CSS

Video:

HTML/CSS + animation + dynamic media

↓

HyperFrames

↓

MP4

---

## # 34. VIDEO CONTEXT

Не использовать один статичный арт на весь ролик.

Если Story про:

DLSS

можно использовать:

- game footage;
- GPU image;
- DLSS screenshots;
- official trailer fragment;
- animated UI elements.

Но только assets, которые разрешено использовать системой/пользователем.

Создать Asset model:

```
{  
  "asset_id": "...",  
  "type": "image|video|logo",  
  "url": "...",  
  "local_path": "...",  
  "source": "...",  
  "license": "...",  
  "width": ...,  
  "height": ...,  
  "score": ...  
}
```

---

## # 35. DATABASE

PostgreSQL.

Минимальные таблицы:

sources  
feeds  
articles  
facts  
entities  
stories  
story\_sources  
translations

content  
quality\_checks  
assets  
story\_assets  
render\_jobs  
outputs  
pipeline\_runs

Использовать SQLAlchemy.

Миграции:

Alembic.

---

## # 36. CACHE

Обязательно кэшировать:

- downloaded pages;
- extracted articles;
- translations;
- embeddings;
- image downloads;
- image hashes;
- CLIP embeddings;
- generated content.

Повторный запуск не должен снова обрабатывать одинаковые данные.

---

## # 37. JOB SYSTEM

Не делать сложный distributed system в первой версии.

Прототип:

Python async workers.

Варианты:

A:  
asyncio

B:  
Redis + RQ

C:  
Celery

D:  
Dramatiq

Сравнить.

Для MVP выбрать самое простое решение.

---

# 38. API

FastAPI.

Endpoints:

POST /sources  
GET /sources

POST /collect

GET /articles

GET /stories

GET /stories/{id}

POST /stories/{id}/process

POST /stories/{id}/render/card

POST /stories/{id}/render/video

POST /batch

POST /single-news

GET /outputs/{id}

GET /health

GET /metrics

---

## # 39. CLI

Сделать CLI:

news-engine collect

news-engine process

news-engine cluster

news-engine translate

news-engine quality

news-engine assets

news-engine render-card

news-engine render-video

news-engine batch

news-engine single

news-engine benchmark

---

## # 40. CONFIG

Все движки выбирать через config.yaml.

Например:

collector:  
 provider: miniflux

extraction:  
 provider: trafilatura

clustering:  
 provider: allsikt

translation:  
 provider: opus\_mt

summary:  
 provider: textrank

russian:  
 provider: rules

quality:  
 provider: ru\_score

ai\_detector:  
 provider: rubert\_tiny

images:  
 provider: searxng

image\_similarity:  
 provider: clip

renderer:  
 provider: html

video:  
 provider: hyperframes

llm:  
 enabled: true  
 provider: ollama  
 model: ...

---

## # 41. PLUGIN INTERFACES

Каждая функция должна иметь adapter interface.

Например:

Collector  
Extractor  
Deduplicator  
Clusterer  
FactExtractor  
ImportanceScorer  
Summarizer  
Translator  
TerminologyEngine  
EditorialEngine  
QualityChecker  
AIDetector  
ImageSearcher

ImageRanker  
Renderer  
VideoRenderer  
LLMProvider

Не связывать модули напрямую.

---

## # 42. BENCHMARK FRAMEWORK

Это обязательная часть проекта.

Создать:

benchmarks/

С тестовым corpus:

100  
500  
1000  
5000 articles

Сохранить benchmark results в JSON.

Измерять:

latency  
throughput  
RAM  
CPU  
GPU  
LLM calls  
success rate  
quality  
accuracy

---

## # 43. A/B ПРОТОТИПЫ

Обязательно сделать возможность запускать:

Pipeline A

Miniflux

+

Trafilatura

+

Natasha

+

Allsikt

+

TextRank

+

OPUS-MT

+

rules

Pipeline B

Direct RSS

+

readability

+

Natasha

+

E5 embeddings

+

TextRank

+

Argos

Pipeline C

Direct RSS

+

Trafilatura

+

small embedding

+

small local LLM

+

OPUS-MT

И сравнивать.

---

## # 44. ОСНОВНАЯ МЕТРИКА

Не выбирать компоненты только по качеству.

Главная objective function:

## QUALITY / RESOURCE COST

Например:

$$\text{score} = \frac{\text{quality}}{(\text{latency} + \text{RAM penalty} + \text{GPU penalty} + \text{LLM calls penalty})}$$

Отдельно показывать:

quality  
speed  
RAM  
CPU  
GPU  
LLM usage

---

## # 45. GOLDEN DATASET

Создать:

tests/golden/

Минимум 100 реальных игровых новостей.

Для каждой вручную определить:

- правильный cluster;
- важность;
- ключевые факты;
- правильные термины;
- хороший русский вариант;
- допустимые изображения.

Это используется для автоматического сравнения.

---

## # 46. TEST CASES

Обязательно протестировать:

1. одна новость;
2. одна новость из 5 источников;
3. две разные новости об одной игре;
4. одинаковая новость с разными заголовками;
5. статья без RSS полного текста;
6. статья с paywall;
7. статья с плохим HTML;
8. английская статья;
9. японская статья;
10. статья с игровым жаргоном;
11. конфликтующие источники;
12. старое событие;
13. очень важное событие;
14. duplicate image;
15. плохое изображение;
16. image с текстом;
17. лицо в центре;
18. объект слева;
19. объект справа;
20. несколько новостей в carousel.

---

## # 47. OBSERVABILITY

Каждый pipeline run должен иметь:

run\_id

и лог:

SOURCE

→ FETCH

→ EXTRACT

→ DEDUP

→ CLUSTER

→ FACT

→ TRANSLATE

→ EDITORIAL

→ QUALITY

→ ASSET

→ RENDER

Для каждого stage:

duration  
status  
errors  
provider  
model  
cache\_hit

---

#### # 48. FALLBACK

Каждый внешний компонент должен иметь fallback.

Например:

Translation:

OPUS-MT

→ Argos

→ Ollama

Image search:

SearXNG

→ official source

→ alternative provider

Extraction:

Trafilatura

→ readability

→ raw HTML fallback

Summary:

TextRank

→ local LLM

Editorial:

rules

→ local LLM

---

#### # 49. LICENSE CHECK

Перед включением каждого OSS-проекта проверить:

- license;
- commercial use;
- model license;
- redistribution;
- Docker image;
- dependency license.

Создать:

docs/DEPENDENCIES.md

с таблицей:

|                         |
|-------------------------|
| project                 |
| version                 |
| license                 |
| purpose                 |
| runtime                 |
| RAM                     |
| GPU                     |
| commercial restrictions |

---

## # 50. НЕ ДЕЛАТЬ

Не писать собственный:

- RSS parser;
- HTML article parser;
- translation engine;
- NLP engine;
- embedding model;
- image hash;
- video encoder;
- HTTP server;
- database ORM;
- queue system.

Если зрелый OSS уже решает задачу — использовать его.

---

## # 51. НЕ ДЕЛАТЬ СЛИШКОМ РАНО

Не добавлять на MVP:

- Kubernetes;
- Kafka;
- Elasticsearch;
- огромную vector DB;
- микросервисную архитектуру;
- distributed GPU;
- сложный agent framework.

Начать с одного Python backend + PostgreSQL + Docker Compose.

---

## # 52. DOCKER

Создать docker-compose.yml.

Минимально:

postgres  
news-engine

Опционально:

miniflux  
searxng  
ollama  
redis

Но Ollama/Redis не должны быть обязательными для базового pipeline.

---

## # 53. MVP

Первый рабочий MVP должен уметь:

RSS

- article extraction
- dedup
- story clustering
- facts
- importance
- translation
- terminology
- Russian editorial checks
- quality score

- image search
- image dedup
- image ranking
- Story JSON
- Telegram text
- VK text
- Card PNG

И работать БЕЗ LLM.

---

#### # 54. MVP-2

Добавить:

- local LLM fallback
- +
- AI text detector
- +
- better summary
- +
- better editorial repair.

---

#### # 55. MVP-3

Добавить:

- VK carousel
- +
- HyperFrames
- +
- dynamic MP4
- +
- asset video search.

---

#### # 56. ОЖИДАЕМЫЙ RESULT

После реализации я хочу получить:

1. working repository;
2. Docker Compose;
3. FastAPI;
4. CLI;

5. PostgreSQL schema;
6. Pydantic schemas;
7. plugin interfaces;
8. adapters;
9. benchmark framework;
10. golden dataset framework;
11. example config;
12. example pipeline;
13. documentation;
14. benchmark report.

---

## # 57. ПОРЯДОК РАБОТЫ OPENCODE

НЕ начинай сразу писать весь код.

Этап 1:

Исследуй OSS-компоненты.

Этап 2:

Составь таблицу:

| FUNCTION       |
|----------------|
| OPTION A       |
| OPTION B       |
| OPTION C       |
| LICENSE        |
| RAM            |
| CPU            |
| GPU            |
| SPEED          |
| QUALITY        |
| MATURITY       |
| API            |
| DOCKER         |
| RECOMMENDATION |

Этап 3:

Спроектируй архитектуру.

Этап 4:

Создай interfaces/adapters.

Этап 5:

Сделай минимальный vertical slice:

RSS

- article
- Story
- translation
- Russian quality
- JSON

Этап 6:

Добавь benchmark.

Этап 7:

Добавь image pipeline.

Этап 8:

Добавь card renderer.

Этап 9:

Добавь Telegram/VK.

Этап 10:

Добавь HyperFrames.

---

## # 58. КРИТИЧЕСКОЕ ТРЕБОВАНИЕ

Если во время разработки обнаружишь более простой или качественный open-source компонент — НЕ следуй этому ТЗ буквально.

Предложи замену.

Но перед заменой:

1. исследуй проект;
2. проверь актуальность;
3. проверь license;
4. проверь installation;
5. проверь performance;
6. сравни с текущим вариантом;

7. объясни trade-offs.

Цель — не реализовать этот документ любой ценой.

Цель — получить:

FAST

+

CHEAP

+

LOCAL

+

MODULAR

+

HIGH QUALITY

+

LOW LLM DEPENDENCY

---

## # 59. ОСОБЫЙ ФОКУС НА РУССКОМ

Русский editorial pipeline является одним из главных компонентов проекта.

Не делать:

English article

→ LLM

→ Russian post

по умолчанию.

Делать:

English article

→ extraction

→ facts

→ terminology protection

→ MT

→ terminology restoration

→ deterministic Russian editorial processing

→ ru-score

→ AI-text detector

→ optional local LLM repair

→ final Russian text

---

## # 60. ОСОБЫЙ ФОКУС НА ФАКТАХ

Content не является источником истины.

Источник истины:

Source + extracted article + verified facts.

Content является производной.

Нельзя:

Source

→ LLM фантазирует

→ post.

Можно:

Sources

→ facts

→ verified Story

→ formatted content.

---

## # 61. ВЫХОДНЫЕ ДАННЫЕ

Каждый pipeline должен заканчиваться валидным Story JSON.

Пример:

```
{
  "story_id": "exodus-2027",

  "importance": 94,

  "facts": [
    {
      "text": "Exodus выйдет 7 апреля 2027 года",
      "source_id": "source-01"
    }
  ],

  "content": {
    "headline": "Exodus выйдет 7 апреля 2027 года",
    "summary": "Archetype Entertainment назвала дату релиза...",
    "telegram": "...",
    "vk": "..."
  }
}
```

```
},  
  
"quality": {  
  "ru_score": 91,  
  "ai_probability": 0.12  
},  
  
"assets": [  
  {  
    "type": "image",  
    "score": 0.93,  
    "width": 3840,  
    "height": 2160  
  }  
]  
}
```

---

## # 62. ФИНАЛЬНЫЙ АРХИТЕКТУРНЫЙ ПРИНЦИП

Не строить "AI news generator".

Строить:

### # FACT-FIRST NEWS ENGINE

где:

```
SOURCE  
↓  
FACT  
↓  
STORY  
↓  
CONTENT  
↓  
CARD  
↓  
VIDEO
```

а AI/LLM — всего лишь один из заменяемых инструментов внутри отдельных этапов.

Главное ядро должно работать без LLM.

LLM должна улучшать систему, а не быть условием её существования.

---

## # 63. ПЕРВАЯ ЗАДАЧА ДЛЯ OPENCODE

Прежде чем создавать production-код, выполни:

1. Исследуй актуальные репозитории и документацию:

- Miniflux
- FreshRSS
- Trafilatura
- Natasha
- Allsikt
- Summa/TextRank
- CTranslate2
- OPUS-MT
- Argos Translate
- ru-text
- humanizer-ru
- ru-ai-text-detector
- SearXNG
- ImageHash
- CLIP
- HyperFrames

2. Найди возможные современные альтернативы каждому компоненту.

3. Не принимай список выше как окончательный.

4. Для каждого этапа выбери:

- recommended;
- lightweight;
- quality-first.

5. Создай:

docs/ARCHITECTURE.md  
docs/COMPONENT-COMPARISON.md  
docs/BENCHMARK-PLAN.md  
docs/DEPENDENCIES.md

6. После этого реализуй минимальный vertical slice.

7. НЕ реализуй остальные модули до проверки vertical slice.

8. После первого benchmark предложи, какие компоненты оставить.

Главный критерий:

НЕ количество написанного кода.

Главный критерий:

сколько функциональности мы получили из существующего OSS при минимальном количестве собственного кода и минимальном количестве LLM-вызовов.